Distributed Tracing (Micrometer + Open Telemetry)

Lets first understand the difference between Logs and Tracing:

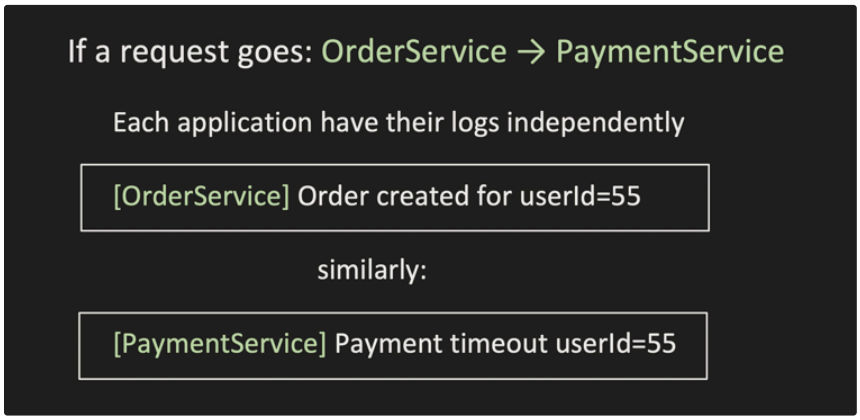
<u>Logs:</u>

- Log contains plain text or JSON to record what events happened during the processing of an API request within the application.
- We can record: Information, Warning or Error, which is crucial for:
  - Debugging, to know what happened within an application while processing the api request.
  - Tracking errors and failures with stack trace.
  - Monitoring application behavior for unusual pattern.
- Logs can record **API-level details**, such as:
  - Incoming request information (URL, method, headers, body)
  - Events happening while an API request is being processed in an application.
  - Outgoing response details (status code, time taken, payload)

<u>Limitation:</u>

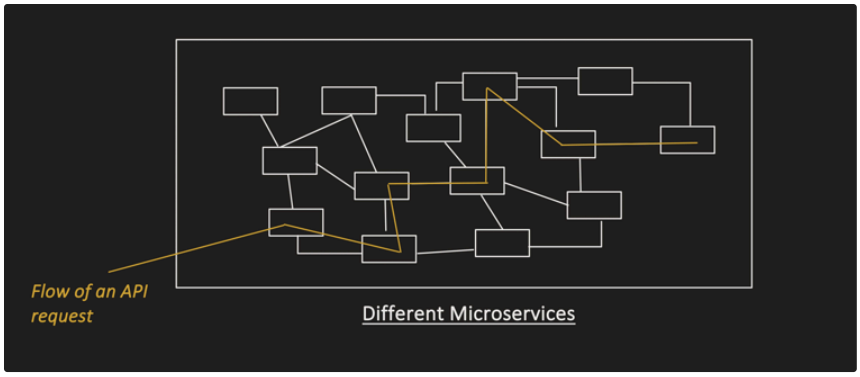
- Logs do not show API request end-to-end journey across microservices.
- We don't know the time breakdown per services, which helps us to know where the request spent more time.

Concept && Coding



But we can't easily tie them together (like both these logs belongs to same API request or not) or how long each step took, that also we don't know.

<u>That where, Distributed Tracing comes into the picture:</u>



- Tracing tracks the path of a request across microservices.
- Shows high-level **info**, like:
  - Which service called which service.
  - How much time each service took.
  - Status code (200, 400, 500) of each service.

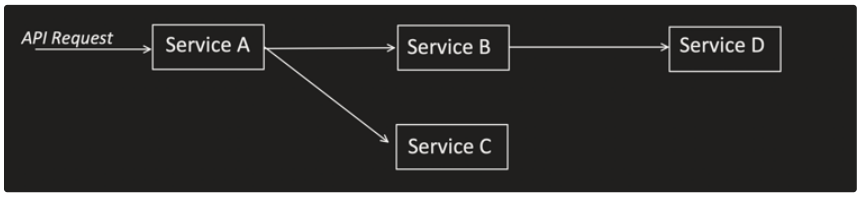
    etc.
- But it does NOT store detailed logs.

To achieve this, Distributed Tracing creates **TRACE_ID** and **SPAN_ID**

trace ID : connect request across microservices with 1 unique ID.

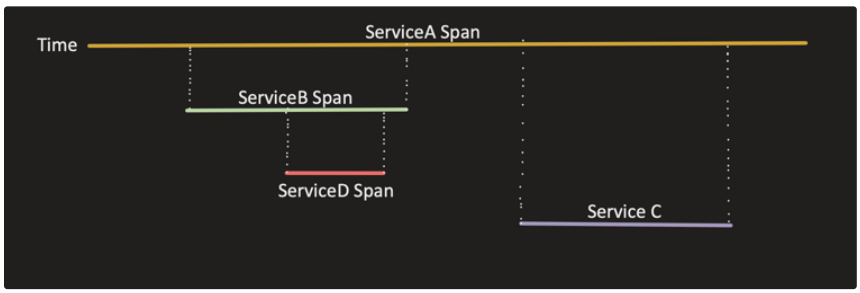
Span IDs: Each service has their unique Span ID, represent individual parts of the work done inside each service.

Concept && Coding



Service	Span_Id	Parent_Span_Id	Trace_Id
Service A	S1	null	T1
Service B	S2	S1	T1
Service C	S3	S1	T1
Service D	S4	S2	T1

Trace_Id common for each service



Note:

Modern distributed Logging system, uses the Trace Id generated by distributed tracing to stich together logs from multiple microservices that belongs to same API request.

If a request goes: OrderService → PaymentService

Each application have their logs independently but stitched with unique and common Trace_Id.

TRACE-ID=T123 [OrderService] Order created for userId=55

similarly:

TRACE-ID=T123 [PaymentService] Payment timeout userId=55

Even though these are separate logs from separate services, they can be combined in Kibana / CloudWatch etc.

Like When we search using traceId = T123, all related logs from different services involved in a single API request appear together.v

Will see this *Distributed Logging* in next topic.

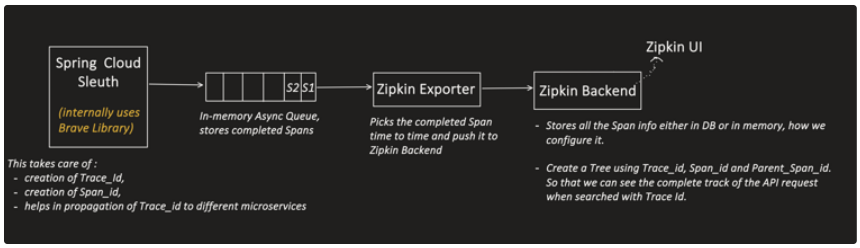
But today lets focus on Distributed Tracing and its implementation.

Now, we know what is Distributed Tracing, Lets see how it works:

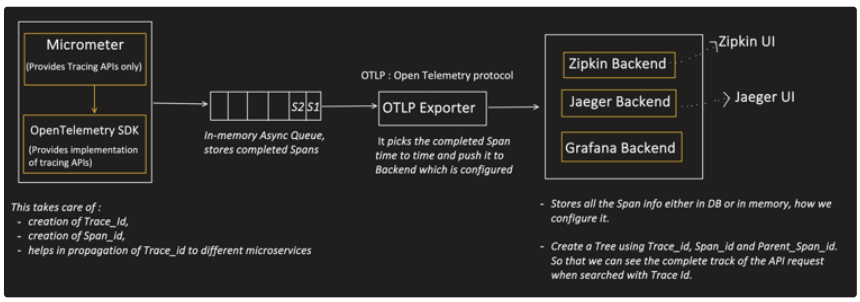
Concept && Coding

<u>Old/Legacy flow:</u>

- Uses Spring Cloud Sleuth, which uses brave library which is compatible with Zipkin backend service.
- So supporting other backend service like Jaeger, its not straight forward. (need some configuration hacks).



<u>Modern flow:</u>



Implementation:

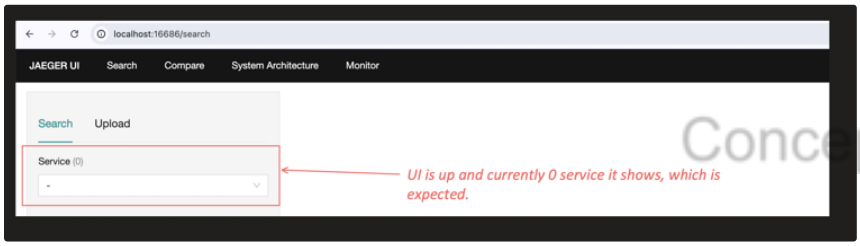
Step1: Choose Backend Service and make their server up

I am going with Jaeger: We can use docker to install and run it:

```
docker run -d -p 16686:16686 -p 4317:4317 -p 4318:4318 jaegertracing/all-in-one:latest
```

Docker does the following steps:

1. Check if image exists locally
2. If not, pulls the image from Docker Hub
3. Creates a container
4. Runs it in the background (-d)
5. Exposes ports
   - 16686 → Jaeger UI (open browser: http://localhost:16686)
   - 4317 → OTLP gRPC endpoint
   - 4318 → OTLP HTTP endpoint



Concept && Coding

Step2: Application1 setup

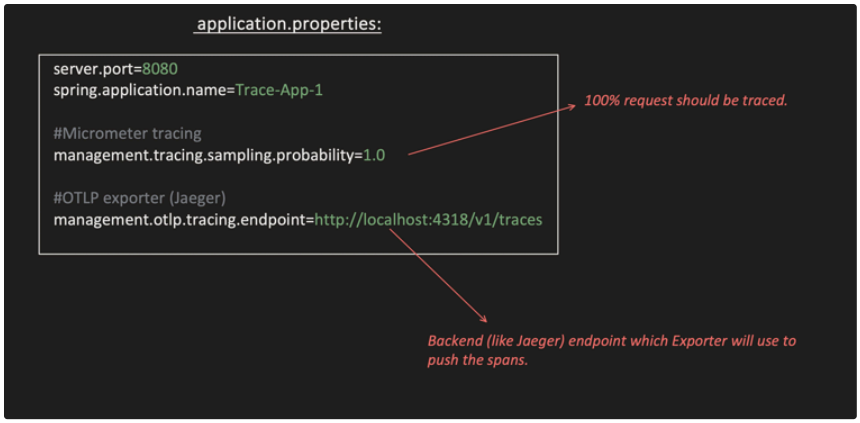
pom.xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>

<dependency>
    <groupId>io.opentelemetry</groupId>
    <artifactId>opentelemetry-exporter-otlp</artifactId>
</dependency>
```

With Actuator, internally it brings Micrometer dependency too (which has Interfaces, not implementation)

OpenTelemetry SDK, implementation of Micrometer APIs

OpenTelemetry Exporter, to push the Spans to backend like Jaeger.



pom.xml (Code)

```
<dependency>
         <groupId>org.springframework.boot</groupId>
         <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
        <groupId>io.micrometer</groupId>
        <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>

<dependency>
       <groupId>io.opentelemetry</groupId>
       <artifactId>opentelemetry-exporter-otlp</artifactId>
</dependency>

```

application.properties (code)

Concept && Coding

```
server.port=8080
spring.application.name=Trace-App-1

#Micrometer tracing
management.tracing.sampling.probability=1.0

#OTLP exporter (Jaeger)
management.otlp.tracing.endpoint=http://localhost:4318/v1/traces

```

That's all we need:

Now tracing automatically initializes and starts generating spans for:

- incoming HTTP requests
- Outgoing HTTP requests
- thread handoffs

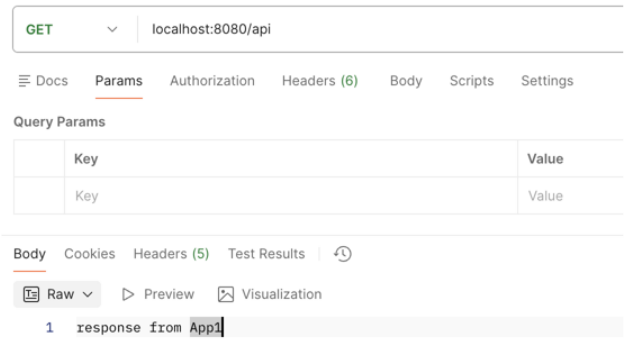
  etc.

Lets add controller, just for testing :

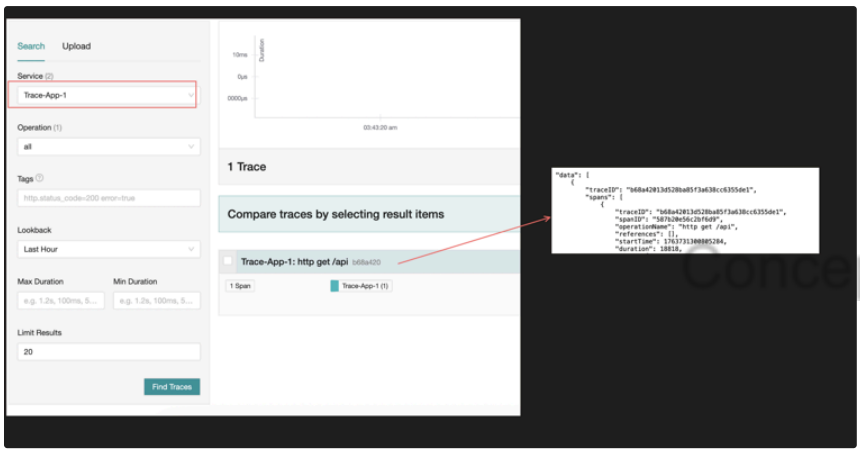
```
@RestController
public class App1Controller {

@GetMapping("/api")
public String invokeApp2() {
             //here will be invoking app2 microservices
```

```
            return "response from App1";
    }
}
```

When we will invoke this endpoint, it should generate the trace_id and span_id for the incoming Http request.



Jaeger UI:



Step3: Application2 setup

pom.xml

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
       <groupId>io.micrometer</groupId>
       <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>

<dependency>
      <groupId>io.opentelemetry</groupId>
      <artifactId>opentelemetry-exporter-otlp</artifactId>
</dependency>
```

application.properties

```
server.port=8082
spring.application.name=Trace-App-2
```

```

#Micrometer tracing
management.tracing.sampling.probability=1.0

#OTLP exporter (Jaeger)
management.otlp.tracing.endpoint=http://localhost:4318/v1/traces

```

Lets add controller just for testing:

```
@RestController
public class App2Controller {

  @GetMapping("/api/hello")
  public String helloMethod() {
                return "hello from App2";
    }
}

```

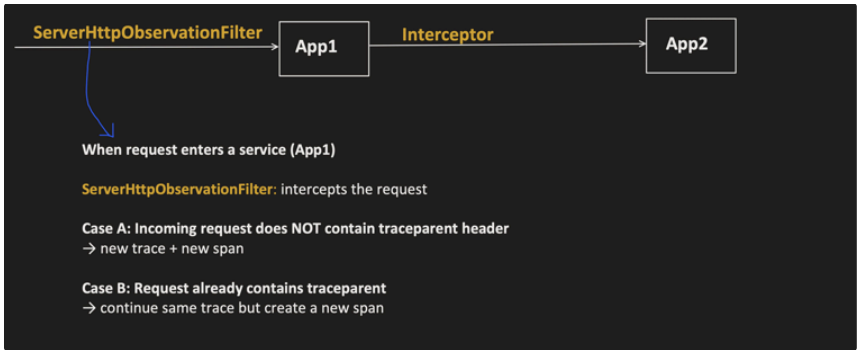
Step4: Application1 need to invoke Application2

Application1 Controller (update to invoke the Application2 service):

```
@RestController
public class App1Controller {

@Autowired
private RestClient restClient;

@GetMapping("/api")
public String invokeApp2() {

            String response = restClient.get()
                .uri("http://localhost:8082/api/hello")
                .retrieve()
                .body(String.class);

                 return "Response from App2:" + response;
   }
}

```

Concept && Coding

```
@Configuration
public class AppConfig {

@Bean
RestClient restClientInstance(RestClient.Builder builder) {
                return builder.build();
     }
}

```



Now with this, during Outgoing call, it will start propagating trace id and spans to other microservices. like below will get appended in header:

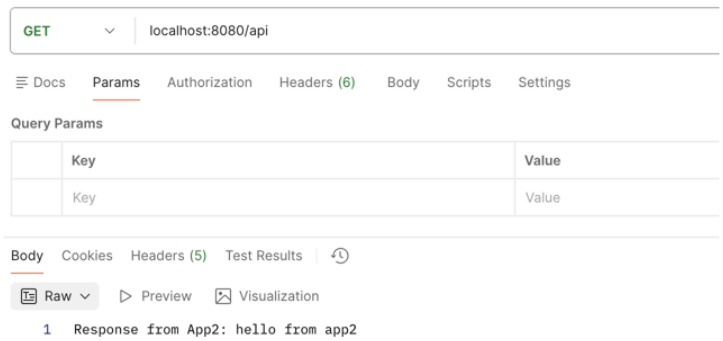
header Name = <trace id>-<span id>

GET /api/hello HTTP/1.1
header:
traceparent: 31b2e6bc8d9e102a84b29c7f1e568888-72fd8c9d4d47e352

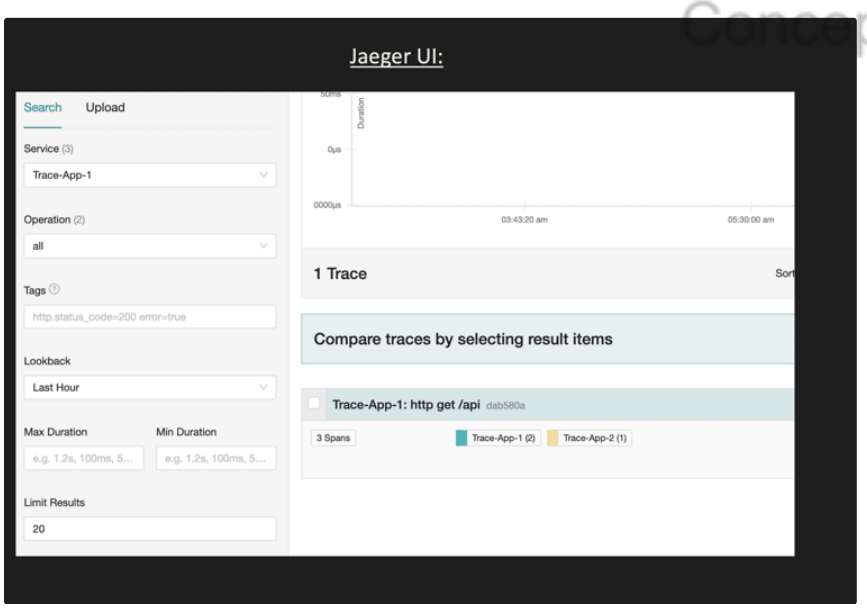
Http Client	Need Customizer	
RestTemplate	Yes *@Bean* *RestTemplate restTemplate(ObservationRestTemplateCustomizer customizer){* *RestTemplate restTemplate = new RestTemplate();* *customizer.customize(restTemplate);* *return restTemplate;* *}*	Old way of invoking the API, so customizer adds the interceptor, which adds trace_id and span_id to header.
RestClient	NO: When used RestClient.Builder, Springboot automatically adds ObservationRestClientCustomizer (as it can detect micrometer dependency on the classpath) *@Bean* *RestClient restClientInstance(RestClient.Builder builder){* *return builder.build();* *}*	Built in support with RestClient.Builder

Concept && Coding

	Yes: RestClient restClient = RestClient.create(); Now, trace id propagation will not work. As interceptor is not added automatically.	
FeignClient	No *Auto configuration present, Test it out please.*	Built in support



Concept && Coding



Above Span in code form:

```
{
    "data": [
        {
            "traceID": "dab580a0614028158d96fa5dded6b2b3",
            "spans": [
                {
                    "traceID": "dab580a0614028158d96fa5dded6b2b3",
                    "spanID": "6571f74f2fdf7017",
                    "operationName": "http get /api",
                    "references": [],
                    "startTime": 1763735113048936,
                    "duration": 50859,
                 "processID": "p1"
                },
                {
                    "traceID": "dab580a0614028158d96fa5dded6b2b3",
                    "spanID": "01b20cd4bed74e03",
                    "operationName": "http get",
                    "references": [
                        {
                            "refType": "CHILD_OF",
                            "traceID":
"dab580a0614028158d96fa5dded6b2b3",
                            "spanID": "6571f74f2fdf7017"
                        }
                    ],
                    "startTime": 1763735113069676,
                    "duration": 24497,
                  "processID": "p1"
                },
                {
                    "traceID": "dab580a0614028158d96fa5dded6b2b3",
                    "spanID": "fb9739086e3ebf65",
                    "operationName": "http get /api/hello",
                    "references": [
                        {
                            "refType": "CHILD_OF",
                            "traceID":
"dab580a0614028158d96fa5dded6b2b3",
                            "spanID": "01b20cd4bed74e03"
                        }
```

Concept && Coding

```
                    ],
                    "startTime": 1763735113086468,
                    "duration": 1193,
                "processID": "p2"
                }
            ],
            "processes": {
                "p1": {
                    "serviceName": "Trace-App-1"
                },
                "p2": {
                    "serviceName": "Trace-App-2"
                }
            }
        }
    ]
}

```

Manual Span Creation:

Concept && Coding

```
@RestController
public class App1Controller {

    @Autowired
    Tracer tracer;

     @GetMapping("/api/test/span")
    public String manualSpanTest() {

        //creating a new span here:
        /* this will return me the current span ongoing */
        Span parentSpan = tracer.currentSpan();

        /* create a child span under same trace id */
```

```java
        Span childSpan = tracer.nextSpan(parentSpan).name("manual-child-span");
        //starts the timer, but does not activate the span as current span (current span is still parent only)
        childSpan.start();

        Tracer.SpanInScope spanInScope = tracer.withSpan(childSpan);
        try {
            // business logic
            Thread.sleep(70);
        } catch (Exception ex) {
            //handle exception here
        } finally {
            if (spanInScope != null) {
                spanInScope.close();    // restore parent span as current
            }
            childSpan.end(); //stops the timer
        }

        return "Response from App1";
    }
}

```

Span info in Jaeger (in JSON form):

Manual span with "spanID": "137b2c47fe0e0949" is child of "spanID": "64dd4cbf327312fc",

Concept && Coding


```json
{
    "data": [
        {
            "traceID": "96420ca9ec858978ad6db56830f26899",
            "spans": [
                {
                    "traceID": "96420ca9ec858978ad6db56830f26899",
                    "spanID": "64dd4cbf327312fc",
                    "operationName": "http get /api/test/span",
                    "references": [],
                    "startTime": 1763745034193419,
                    "duration": 1720669,
                    "processID": "p1"
                },
                {
                    "traceID": "96420ca9ec858978ad6db56830f26899",
                    "spanID": "137b2c47fe0e0949",
                    "operationName": "manual-child-span",
                    "references": [
                        {
                            "refType": "CHILD_OF",
                            "traceID": "96420ca9ec858978ad6db56830f26899",
                            "spanID": "64dd4cbf327312fc"
                        }
                    ],
                    "startTime": 1763745034194449,
                    "duration": 1578050,
                    "processID": "p1"
                }
            ],
            "processes": {
                "p1": {
                    "serviceName": "Trace-App-1"
                }
            }
        }
    ]
}

```

Concept && Coding